

Multi-Node Linux Infrastructure on AWS

AWESOME CYBER PROS

Ocala, FL

William Selk

AWESOME CYBER PROS | ORLANDO INT WAY 5555, ORLANDO FL 32801

Table of Contents

Project Overview	2
Project Goals & Objectives	2
Network Infrastructure Requirement	3
Assumptions & Risks.....	3
Network Configuration & Procedural Checklist.....	4
Network Topology Diagram with IP Addresses	5
Appendices Deliverables.....	5

Scope of Work

Portfolio 5

Cloud hosted with load balancing

Project Overview

The scope of this project is to design and demonstrate a cloud-hosted SeedDMS system using Amazon Web Services. The work includes setting up the core server components, configuring basic network access, and enabling secure communication through SSL encryption. Multiple application servers will be used to show how availability and load distribution can be handled in a real-world style environment, while a central database will store shared documents and metadata. The project focuses on illustrating concepts discussed in class, such as cloud hosting, security basics, and system organization, rather than implementing a fully production-ready solution. Advanced features such as automated scaling, full disaster recovery planning, and long-term monitoring are considered outside the scope of this project.

Milestone completion dates:

Milestone I – December 3, 2025

Milestone II – December 8, 2025

Milestone III – December 11, 2025

Milestone IV – December 18, 2025

Completion – Dec 21, 2025

Project Goals & Objectives

After implementation, Awesome Cyber Pro's will setup a centralized and more secure way to manage documents through the SeedDMS platform. The network is designed to support reliable access for users while maintaining acceptable performance during periods of higher activity. Basic redundancy and cloud hosting help reduce the impact of individual system failures, contributing to overall system reliability. The design also allows for future expansion, supporting additional users or resources as needed. These goals align with course concepts related to secure, scalable, and efficient network design.

Network Infrastructure Requirement

The SeedDMS website is hosted on Amazon Web Services using a virtual network that keeps public access separate from internal systems. Users connect through a single-entry point, which helps manage traffic and ensures the site remains available even if one server experiences issues. Behind the scenes, multiple web servers handle requests while a centralized database stores shared data. This setup reflects how many real-world cloud applications are deployed today, offering a balance between reliability, security, and cost without requiring complex physical infrastructure.

Assumptions & Risks

Assumptions

- Amazon Web Services will provide reliable cloud infrastructure and availability for hosting the SeedDMS environment.
- Internet connectivity for end users will be stable enough to allow consistent access to the system.
- Required system updates and routine maintenance will be performed in a timely manner to keep the environment secure and operational.
- Awesome Cyber Pros and (Organization) will perform mandatory training to spin up all parties on network implementation.

Risks

- If system usage grows faster than expected, performance may be affected until additional resources are provided.
- Misconfiguration or delayed updates can cause service interruptions.
- Failure of a virtual server or supporting cloud component could temporarily reduce system availability despite redundancy.
- Future feature requests or compliance requirements may exceed the scope of the current design and require additional planning.
- Ongoing operational costs may increase over time as storage needs and system usage expand.

Network Configuration & Procedural Checklist

Deploy Operating Systems

- Install Linux on all EC2 instances (HAProxy, SeedDMS servers, Database)
- Document OS version and any initial updates applied

Groups & Permissions

- Create groups for administrative and standard users
- Assign users to appropriate groups
- Document permissions assigned to each group

Folders & File Permissions

- Create directories for SeedDMS application data, logs, and backups
- Set folder permissions according to user groups
- Note which directories are shared vs. private

Network Configuration

- Assign private IPs to SeedDMS servers and database
- Configure HAProxy as the single public-facing entry point
- Document firewall/security group rules (ports allowed, restrictions)

Security Measures

- Enable SSL encryption for web access
- Confirm internal servers are not directly accessible from the internet
- Document security settings and any applied updates

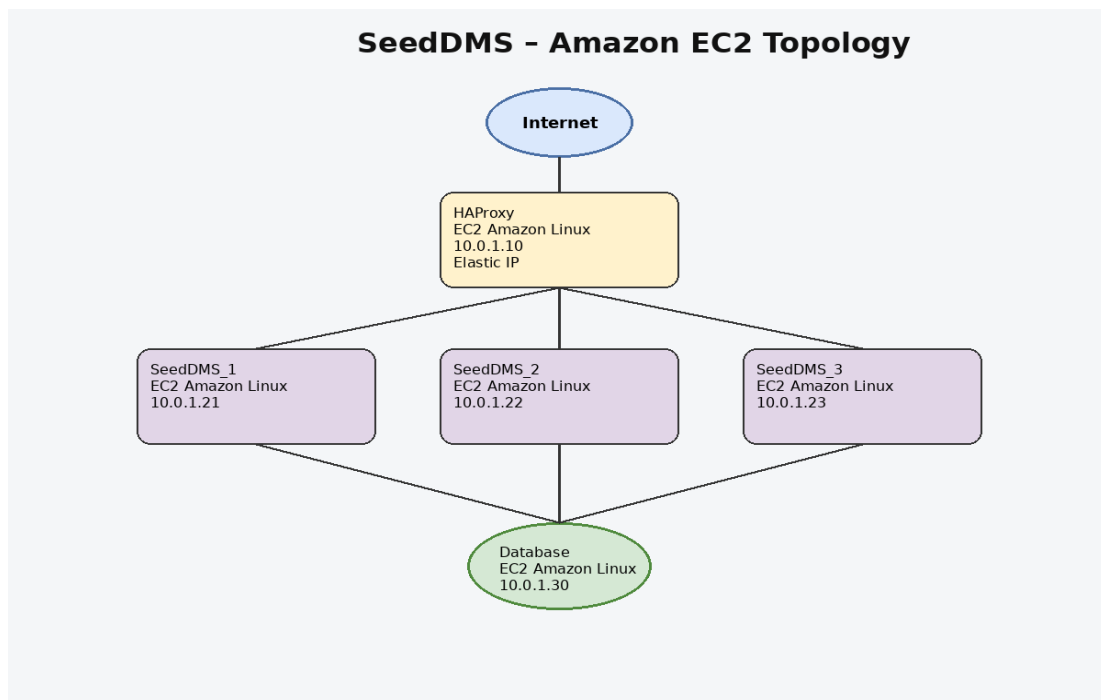
Database Setup

- Install and configure central database for SeedDMS
- Ensure all application servers can connect securely
- Document database credentials (or note default credentials changed)

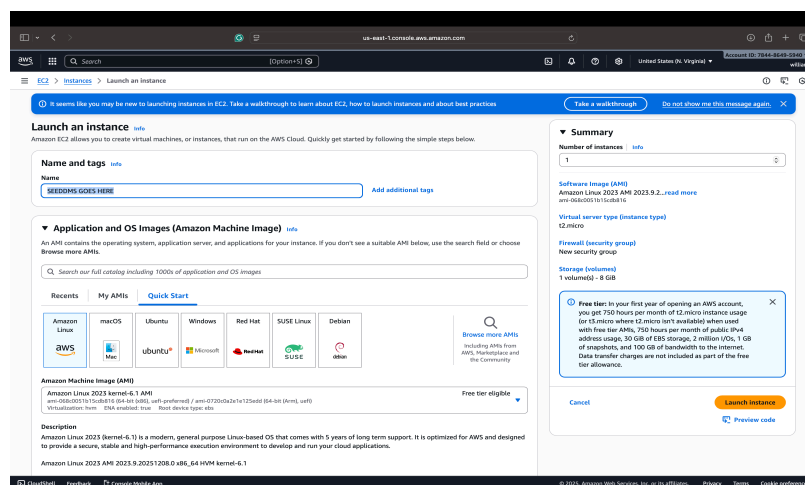
Testing & Verification

- Test user logins and permissions
- Test HAProxy load balancing between SeedDMS servers
- Verify database connectivity and data access

Network Topology Diagram with IP Addresses

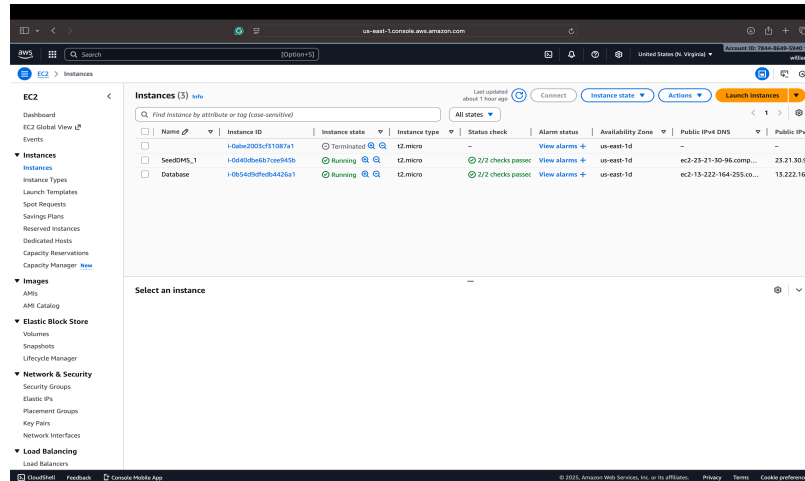


Appendices | Deliverables

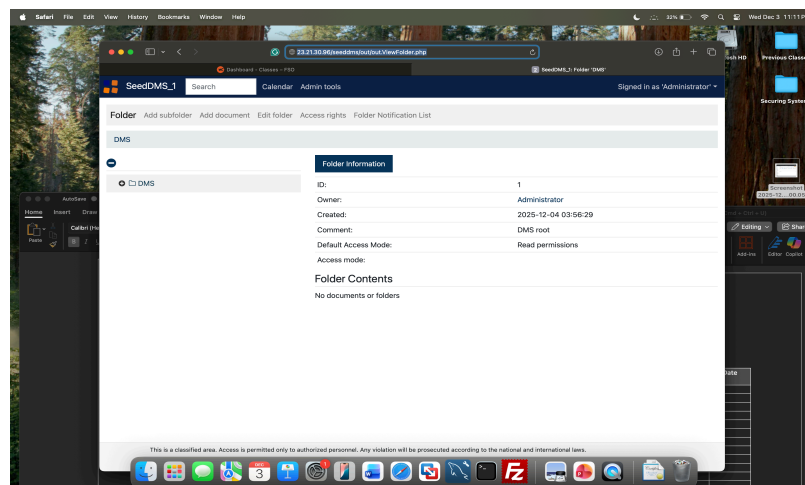


This screen grab shows the EC2 instance where we go in and select what we want to make or “launch”. We select the operating system and basic settings for the instance. It is the starting point for getting our servers ready so the SeedDMS application can be installed. Here we also

made a key pair to share with the other VM's and created the security group and named it portfolio 5 to attach the rest of the instances to.



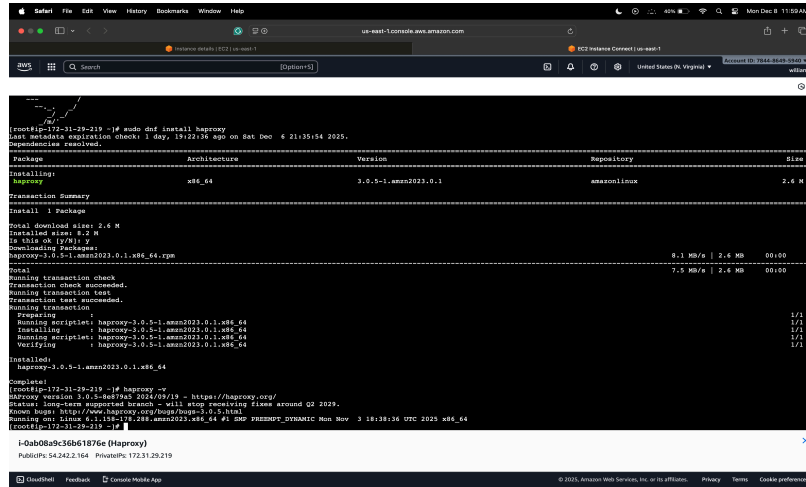
Once we were done with our initial instances we had created the database server and our SeedDMS_1. To get our SeedDMS_2 & _3 I cloned _1 using the AWS built in tools. Creating an AMI is only a few clicks away and once it boots up it's copied all of our relevant data like the updates, modules and files.



This is simply the splash/confirmation screen that we had accurately gotten SeedDMS_1 up and running (for the most part). We had to perform a slew of commands.

- `sudo yum install httpd -y`
- `sudo yum install mariadb-server -y`
- `sudo systemctl enable httpd`

- `sudo systemctl start httpd`
- `sudo systemctl enable mariadb`
- `sudo systemctl start mariadb`



Here we are on our load balancing server setting up haproxy. It is a relatively straight forward process, but we did have to come back in several times throughout the month making updates as our initial setup was too vague and didn't fit specifics like an SSL cert.

- Sudo dnf haproxy
 - Haproxy -v
- Sudo nano /etc/haproxy/haproxy.cfg
 - sudo systemctl enable haproxy
 - sudo systemctl start haproxy


```
defaults
    log global
    mode http
    option httplog
    option dontlognull
    option forwardclose
    option http-server-close
    timeout connect 1m
    timeout client 1m
    timeout server 1m

#-----
# Redirect HTTP (80) -> HTTPS (443)
#-----
frontend http_frontend
    bind *:80
    mode http
    redirect scheme https code 301 if (!ssl_fc)

#-----
# HTTPS Frontend with SSL Termination
#-----
frontend https_frontend
    bind *:443 ssl crt /etc/ssl/certs/haproxy.pem
    mode http
    default_backend webbackends

#-----
# Backend: Your 3 seedbox servers
# Include session stickiness + health checks
#-----
backend webbackends
    mode http
    balance roundrobin
    cookie SERVERID insert indirect nocache
    server seed1 172.31.27.48:80 check cookie seedID_1
    server seed2 172.31.29.48:80 check cookie seedID_2
    server seed3 172.31.29.140:80 check cookie seedID_3
```

For the last shot I included one of my final screen grabs from the term. In here we can see the haproxy.cfg. In here is where everything pertinent to the script lives. What we made the most adjustments to was the backend or rather the last part of the script. We need to make sure roundrobin is enabled and that cookies are used and the IP's were all matchd up.

- `sudo nano /etc/haproxy/haproxy.cfg`
balance roundrobin
- cookie SERVERID insert indirect nocache
- server seed1 10.0.0.10:80 check cookie s1
- server seed2 10.0.0.11:80 check cookie s2
- server seed2 10.0.0.12:80 check cookie s3